How to call SIF API in 360° Online

Table of Contents

Changelog

Date	Changed by	Comment
2017-11-01	Gry Siri Eggen	Document created
2018-01-11	Gry Siri Eggen	updated with complete url and operation details
2018-02-15	Gry Siri Eggen	Updated with common error messages and renamed from SIF GWSL to SIF API
2018-10-25	Gry Siri Eggen	Added info about Swagger and Postman examples
2019-06-03	Michael Andre Krog	Added info about passing auth key in header
2019-10-04	Michael Andre Krog	Added another common error message
2020-03-31	Michael Andre Krog	Added info about passing client ID (from 360 version 5.5)

1 Introduction

With 360° Online, we no longer support SOAP web services. SIF Generic web service layer has therefore been written as a RPC service. All the objects and functionality are the same as before – you just have to call the new endpoints over the HTTP protocol using POST instead and put the objects as json in the body.

2 General structure

2.1 Calling the service

You will get a URL and a secret authorization key that you need to use to call the service. The DNS name and auth key will differ from customer to customer.

The URL is:

```
https://[customer specific
url]/Biz/v2/api/call/SI.Data.RPC/SI.Data.RPC/[service]/
[operation]?authkey=[auth key]
```

A real example for calling the CreateCase operation in the CaseService on a test-instance of 360:

https://test-integrations-no.public360online.com/Biz/v2/api/call/SI.Data.RPC/SI.Data.RPC/CaseService/CreateCase?authkey=key

2.2 Passing of auth key

There are two ways of passing the auth key for SIF RPC. It can be defined as a query parameter directly in the endpoint/URL that you call (which is mentioned above), or it can be added to the header. Examples can be found below.

2.2.1 URL

The easiest way to authenticate towards SIF RPC, is probably to send the auth key directly in the URL.

However, authentication of the API this way is not the safest alternative, since it's more likely for the auth keys to get unintentionally distributed. For instance, when sending endpoints around by mail, or referencing to stack traces where the endpoints (with the auth key) are included.

Example:

https://test-integrations-no.public360online.com/Biz/v2/api/call/SI.Data.RPC/SI.Data.RPC/SupportService/GetSIFVersion**?authkey=key**

2.2.2 Header

A more robust and secure way of passing the authentication (auth key) to SIF RPC, is to define it in the header rather than directly in the URL.

Example:

https://test-integrations-no.public360online.com/Biz/v2/api/call/SI.Data.RPC/SI.Data.RPC/SupportService/GetSIFVersion

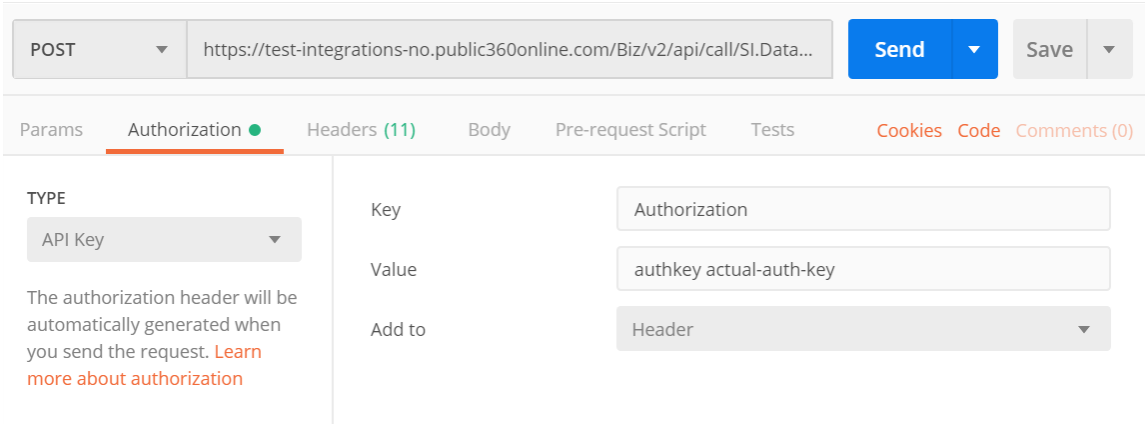
No auth key is passed directly in the URL above, so the auth key must be included in the header. See below for C# and Postman examples.

C# client

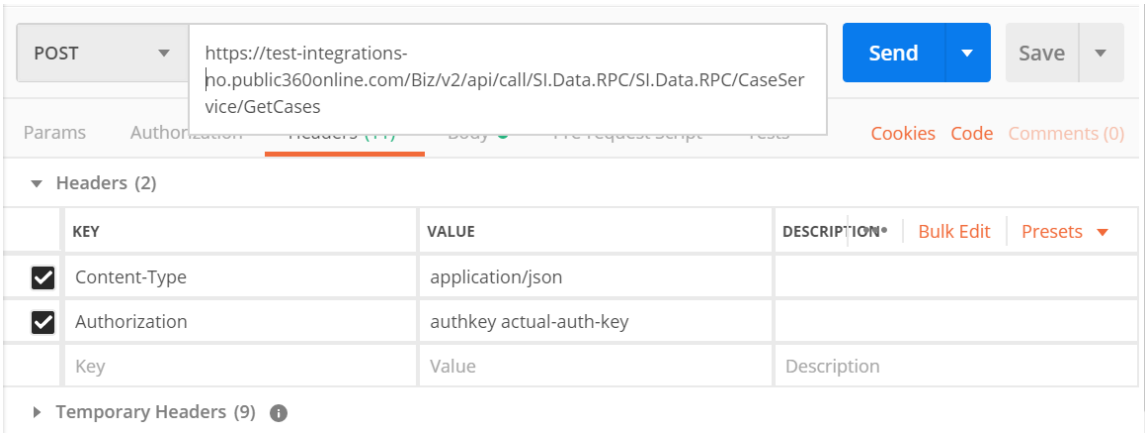
```
using System.Net;
using System.Net.Http;

namespace Test_CallingSIFRPC
{
    public class Program
    {
        static void Main(string[] args)
        {
            string authKey = "just a dummy key";

            var handler = new HttpClientHandler()
            {
                AutomaticDecompression = DecompressionMethods.GZip |
DecompressionMethods.Deflate,
            };

            var client = new HttpClient(handler);

            // This is the code that puts the auth key in the
Authorization header.
            // The format MUST be:
            // authkey ActualAuthKey
            // Prefix "authkey" must be in lowercase letters.
            client.DefaultRequestHeaders.Add("Authorization", "authkey
" + authKey);

            // Rest of the code, calling the API, etc...
        }
    }
}
```


Postman

Alternative 1



Alternative 2



2.3 Passing of client ID

From 360° version 5.5, you can define endpoints where a client ID is required in addition to an auth key. The auth key is something that is generated on the 360° side. However, the client ID is a value that is provided/received by a third party. This way, it's possible to introduce an extra layer of security, a sort of handshake if you will. It also makes the endpoint more robust. If someone gets their hand on the auth key, they still can't authenticate without having the client ID as well.

Below are some quick examples on passing this new property.

2.3.1 URL

Example:

https://test-integrations-no.public360online.com/Biz/v2/api/call/SI.Data.RPC/SI.Data.RPC/SupportService/GetSIFVersion**?authkey=key&clientid=id**

2.3.2 Header

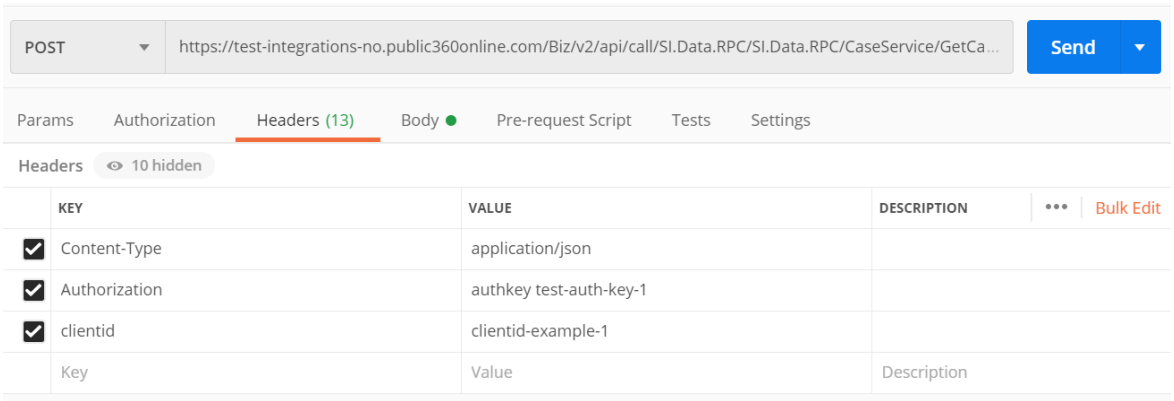
Example:

https://test-integrations-no.public360online.com/Biz/v2/api/call/SI.Data.RPC/SI.Data.RPC/SupportService/GetSIFVersion

C# client

```
//Exactly the same as the previous client code, only with the relevant "clientid" change below

            // The format MUST be as below, with the client ID header in one word (case sensitive):
            client.DefaultRequestHeaders.Add("clientid", clientId);

            // Rest of the code, calling the API, etc...
        }
    }
}
```

Postman



2.4 Going from SOAP to RPC

The services (CaseService, DocumentService and so on) and their operations are the same as the one in the SOAP services, with identical names. Either objects from the SOAP WSDL og the names from the SIF Documentation can be used.

The parameters of the operations need to be sent in the body of the request as json objects. Since it can be several parameters in one request, you first must have one object as a “wrapper” around the parameter object(s):

```
{
   "parameter": {
        "Title": “Testcase 1”,
        "CaseType": “recno:2”
   }
}
```

An example body for creating document (DocumentService/CreateDocument):

```
{
parameter:{
          ADUserContext:"no\\grysiri.eggen",
          Title:"blabla",
          Category:"recno:110",
          Status:"recno:1",
          CaseNumber:"18/00005",

          Files:[
                    {
                    Title:"første fil",
                    Format:"pdf",
                    Base64Data:"JVBERi0xLjQKJeLjz9MKNSAwIG9iago8"
                    }
           ]
      }
}
```

Example of UserService/SyncronizeUser:

```
{
  "parameter": {
    "ADContextUser": "domain\username",
    "Login": "username",
    "ContactExternalId": "1234",
    "IsActive": true,
    "Profiles": [
      {
        "Role": "string",
        "EnterpriseId": "string",
        "FromDate": "2018-01-11T16:07:36.478Z",
        "ToDate": "2018-01-11T16:07:36.478Z"
      }
    ],
    "AccessGroups": [
      {
```

```
      "AccessGroup": "string",
      "FromDate": "2018-01-11T16:07:36.478Z",
      "ToDate": "2018-01-11T16:07:36.478Z"
    }
  ]
 }
}
```


You will get the return objects back in the content, which can be deserialized back to the objects defined in the wsdl of the SOAP web service.

If something goes wrong, you will get an indication of what went wrong in the HTTP response code and error message.

2.5 HTTP codes

When everything goes okay, you will get the HTTP status code 200 OK back. This includes insert, update, sync and get. You will also get a 200 OK back if you are searching for any entity and get zero hits.

If something went wrong in the operation, like if you tried to create a Case with invalid values, you will get a Successful:false.

Example of a success return from SyncronizeUser:

{ "Recno": 200001, "Successful": true, "ErrorMessage": "string", "ErrorDetails": "string" }

Example of an error message:

{"Recno":0,"Successful":false,"ErrorMessage":"Can not inititate ADContextUser because the context user is empty or invalid","ErrorDetails":null}

2.6 Swagger

Swagger can be found here: https://[customer-specific-domain]/Biz/v2/api/swagger/SI.Data.RPC

Replace the domain with the domain to the environment you are using.

Note that it's only possible to call the API through Swagger on 360° version 5.0 and later. You can still use the Swagger-documentation on earlier versions to get an overview of the API. It is possible to view the structure and copy-paste the input-parameters.

2.7 Postman examples

A Postman collection with some examples of SIF RPC-calls can be found at GitHub: https://github.com/SoftwareInnovation/sif-rpc-postman-examples

3 Example in C#

Here is a simple example on how to call the CaseService as RPC

From the wsdl definition of the web service, we see that the CreateCase operation takes in a parameter named “parameter” of the type “CreateCaseParameter”.

```
- <xs:element name="CreateCase">
  - <xs:complexType>
    - <xs:sequence>
        <xs:element name="parameter" type="tns:CreateCaseParameter" nillable="true" minOccurs="0"/>
      </xs:sequence>
    </xs:complexType>
  </xs:element>
```

So we create a body with this filter and pass it on in a POST:

```
var body = new
        {
           parameter = new CreateCaseParameter
           {
              Title= "This is a testcase",
              CaseType="recno:2"
           }
        };
var handler = new HttpClientHandler()
        {
           AutomaticDecompression = DecompressionMethods.GZip |
DecompressionMethods.Deflate,
        };
        var client = new HttpClient(handler);
        var requestcontent = new StringContent(JsonConvert.SerializeObject(body),
Encoding.UTF8, "application/json");
        var result = client.PostAsync(url, requestcontent).Result;
```

From the wsdl we can see that the response of this operation is an object of type CaseOperationResult:

```
- <xs:element name="CreateCaseResponse">
  - <xs:complexType>
    - <xs:sequence>
        <xs:element name="CreateCaseResult" type="q3:CaseOperationResult" nillable="true" minOccu
      </xs:sequence>
    </xs:complexType>
  </xs:element>
```

And we find this in the response like this, for example:

```
var content = result.Content.ReadAsStringAsync().Result;

CaseOperationResult operationResult =
JsonConvert.DeserializeObject<CaseOperationResult>(content);
```

4 Common error messages in SIF RPC

4.1 Index was outside the bounds of the array

Response:

{"Type":"Error","CorrelationId":"a70a91e7-79e1-42fd-9f06-381a61e64e9c","ExceptionType":"IndexOutOfRangeException","Message":"Index was outside the bounds of the array.","Details":null}

Cause: Wrong name of the operation in the call. This is case sensitive, e.g. you will get this error if you try to call Createcase instead of CreateCase

4.2 Missing argument '...'

Response:

{"Type":"Error","CorrelationId":"0fb835f9-9661-43f4-9328-ca63e4c5c274","ExceptionType":"InvalidOperationException","Message":"Missing argument '…'","Details":null}

Cause: You are missing a field in the body. This is also case sensitive.

4.3 401 Unauthorized: Unable to authorize request

Cause: There's an error in the SIF configuration, and the problem is the user the service is running as. Contact Tieto.

Or, the problem could simply be due to a wrong or misspelled auth key.

4.4 200 OK: Can not inititate ADContextUser because the context user is empty or invalid

Cause: There's something wrong with the user in the ADContextUser field in the parameter – either the user is not a valid 360° user or the format of the user's ID is wrong.

4.5 Object reference not set to an instance of an object

Response:

{"Type":"Error","CorrelationId":"d4364ac0-b2f1-4a98-bc48-1901ad2e5e8c","ExceptionType":"NullReferenceException","Message":"Object reference not set to an instance of an object.","Details":null}

Cause: You may experience this error when you are trying to send a request using a HTTP verb other than POST. If GET is defined for example, you must change it to POST, even though the operation might be a *Get* operation. SIF RPC runs only with the POST verb. Another possible reason could be that no body/an empty body is provided in the request.

